

NetSuite Academy — Full Prompt History

Project Initialization

1. Build the NetSuite Academy project for the hackathon.
 2. Create a full-stack learning management platform focused on NetSuite training.
 3. Design the complete architecture for the application.
 4. Use React and TypeScript for the frontend.
 5. Use FastAPI for the backend.
 6. Use PostgreSQL as the database.
 7. Use SQLAlchemy for database operations.
 8. Use JWT authentication.
 9. Create separate learner and admin experiences.
 10. Make the project production-like and suitable for a hackathon demonstration rather than a simple prototype.
-

Backend

1. Set up the FastAPI backend.
2. Create the backend project structure.
3. Configure PostgreSQL.
4. Configure SQLAlchemy.
5. Create the database models.
6. Create the database session and connection handling.
7. Create the FastAPI application entry point.
8. Add CORS configuration so the React frontend can communicate with the backend.
9. Create the API routers.

10. Organize the backend into authentication, courses, quizzes, progress, certificates, and admin functionality.
-

Authentication

1. Implement user registration.
 2. Implement user login.
 3. Add password hashing.
 4. Add JWT token generation.
 5. Add JWT token validation.
 6. Create authenticated-user dependencies.
 7. Add learner/admin roles.
 8. Create an admin authorization dependency.
 9. Protect admin-only endpoints.
 10. Make sure unauthenticated users cannot access protected endpoints.
 11. Make sure learners cannot access admin endpoints.
 12. Create an admin account for testing.
 13. Test admin login through Swagger.
 14. Test JWT authorization through Swagger.
 15. Fix the authentication flow if login returns errors.
 16. Fix password hashing/bcrypt compatibility issues.
 17. Resolve the bcrypt 72-byte password problem.
 18. Test the admin credentials again after changing the bcrypt version.
 19. Verify that the JWT returned by login works with the protected admin APIs.
-

Database

1. Create the User model.
 2. Create the Course model.
 3. Create the Module model.
 4. Create the Quiz model.
 5. Create the QuizQuestion model.
 6. Create the QuizAttempt model.
 7. Create the Progress model.
 8. Create the Certificate model.
 9. Define relationships between users, courses, modules, quizzes, attempts, progress records, and certificates.
 10. Add appropriate primary keys and foreign keys.
 11. Make sure database records persist correctly.
 12. Make sure deleting/updating related records does not break the application.
-

Courses

1. Implement course creation.
2. Implement course listing.
3. Implement published-course retrieval.
4. Implement course details.
5. Implement module creation.
6. Implement module listing.
7. Associate modules with courses.
8. Add module ordering.
9. Add course descriptions.

10. Add module descriptions.
 11. Support video URLs.
 12. Support text learning content.
 13. Allow courses to contain multiple modules.
 14. Allow courses to be published/unpublished.
 15. Make sure learners only see appropriate published courses.
-

NetSuite Learning Content

1. Structure NetSuite training into meaningful courses.
 2. Divide courses into logical modules.
 3. Add real NetSuite training videos.
 4. Add learning descriptions for each video.
 5. Add supporting text content.
 6. Make the training content realistic rather than placeholder content.
 7. Make each module teach a specific NetSuite concept.
 8. Create quizzes relevant to the content of each module.
 9. Use the uploaded training videos as the actual learning material where appropriate.
-

Learner Dashboard

1. Create the learner dashboard.
2. Show available courses.
3. Show enrolled courses.
4. Show course progress.
5. Show completed courses.
6. Show certificates.

7. Add course cards.
 8. Add progress indicators.
 9. Add course completion information.
 10. Make the dashboard visually polished.
 11. Make the learner experience intuitive.
-

Course Learning Page

1. Create the course details page.
 2. Display course information.
 3. Display the list of modules.
 4. Allow the learner to start a course.
 5. Create the module learning interface.
 6. Display module video content.
 7. Display module text content.
 8. Add navigation between modules.
 9. Track the learner's current module.
 10. Show completed modules.
 11. Prevent incorrect progress states.
 12. Make the learning experience responsive.
-

Quiz System

1. Create quiz functionality.
2. Create quiz questions.
3. Support multiple-choice questions.
4. Store correct answers.

5. Allow learners to submit quiz answers.
 6. Calculate quiz scores automatically.
 7. Define a passing score.
 8. Return the quiz result to the learner.
 9. Show whether the learner passed or failed.
 10. Store quiz attempts.
 11. Track the number of attempts.
 12. Allow failed quizzes to be retried.
 13. Make sure successful quiz completion marks the module as completed.
 14. Make sure failed quizzes do not incorrectly mark the module as completed.
 15. Display useful feedback after submission.
 16. Improve the quiz UI.
-

Progress Tracking

1. Implement module progress tracking.
2. Track completed modules.
3. Track incomplete modules.
4. Track quiz attempts.
5. Track scores.
6. Calculate overall course progress.
7. Update progress after successful module completion.
8. Mark a course completed only when all required modules are completed.
9. Display progress on the learner dashboard.
10. Display progress inside the course.
11. Test progress persistence after refreshing the page.

Course Completion

1. Implement course completion logic.
 2. Check whether all modules are complete.
 3. Mark the course as completed.
 4. Record the completion date.
 5. Calculate/store the final score where required.
 6. Trigger certificate creation after course completion.
 7. Make sure certificates are not incorrectly generated for incomplete courses.
-

Certificate System

1. Create the certificate database model.
2. Generate a unique certificate number.
3. Generate certificates after course completion.
4. Use ReportLab to create PDF certificates.
5. Include the learner's name.
6. Include the course name.
7. Include completion date.
8. Include score.
9. Include certificate number.
10. Design the certificate professionally.
11. Create a certificate viewing page.
12. Create a certificate download endpoint.
13. Make sure the generated PDF actually downloads.
14. Open and verify the downloaded PDF.

15. Verify all certificate information.
 16. Investigate and fix the certificate download implementation if it is still using a temporary/placeholder path.
 17. Explore adding certificate verification.
 18. Explore adding a QR code for certificate verification.
-

Admin Dashboard

1. Create the admin dashboard.
 2. Display total courses.
 3. Display active learners.
 4. Display average pass rate.
 5. Display certificates issued.
 6. Add learner statistics.
 7. Add course statistics.
 8. Add progress statistics.
 9. Add quiz activity.
 10. Add certificate activity.
 11. Create an admin user-management page.
 12. Allow admins to view learners.
 13. Display learner information.
 14. Add course-management functionality.
 15. Add module-management functionality.
 16. Make the admin dashboard visually polished.
-

Admin APIs

1. Create the admin dashboard endpoint.
 2. Create the admin users endpoint.
 3. Protect admin endpoints with authentication.
 4. Protect admin endpoints with the admin role.
 5. Test `/admin/dashboard`.
 6. Test `/admin/users`.
 7. Verify unauthorized users receive the correct response.
 8. Verify learners cannot access admin functionality.
-

Frontend/API Integration

1. Connect the React frontend to FastAPI.
2. Create the API service layer.
3. Implement login API integration.
4. Implement registration API integration.
5. Store authentication tokens appropriately.
6. Send JWT tokens with protected API requests.
7. Handle authentication errors.
8. Handle API errors.
9. Handle loading states.
10. Handle empty states.
11. Handle failed requests.
12. Make sure the frontend correctly updates after course completion.
13. Make sure frontend progress matches backend progress.

14. Make sure certificates displayed in the UI correspond to database records.
-

UI/UX Enhancement

1. Improve the overall visual design.
 2. Make NetSuite Academy look like a professional SaaS/LMS application.
 3. Improve the learner dashboard.
 4. Improve course cards.
 5. Improve navigation.
 6. Improve module presentation.
 7. Improve video presentation.
 8. Improve quiz screens.
 9. Improve progress indicators.
 10. Improve certificate presentation.
 11. Improve admin dashboard visuals.
 12. Add responsive design.
 13. Add loading indicators.
 14. Add useful empty states.
 15. Add useful error messages.
 16. Improve spacing, typography, cards, buttons, and layouts.
 17. Make the UI presentation-ready for the hackathon.
-

Editable Financial Data / Backend Work

1. Work with the uploaded sample financial Excel files.
2. Make the backend accept financial files similar to the sample files.
3. Support files containing real financial data.

4. Support slightly different spreadsheet formats.
 5. Avoid hard-coding the exact structure of one sample Excel file.
 6. Make the backend flexible enough to process similar financial spreadsheets.
 7. Create an editable table for the extracted financial data.
 8. Make sure the frontend table can display and edit the imported data.
 9. Test the implementation against multiple sample financial files.
 10. Fix incorrect parsing/mapping when the spreadsheet structure changes.
-

AI Features

1. Identify ways AI could differentiate NetSuite Academy from a standard LMS.
 2. Explore AI-powered adaptive learning.
 3. Design a personalized learning system based on quiz performance.
 4. If a learner fails a quiz, identify the concepts they struggled with.
 5. Generate personalized remedial explanations.
 6. Recommend relevant learning material.
 7. Allow the learner to retry after remediation.
 8. Make AI remediation part of the learning journey.
 9. Keep the AI feature practical and reliable enough for a live hackathon demo.
-

NetSuite-Specific Differentiation

1. Explore creating a NetSuite simulator.
2. Create a safe practice environment for learners.
3. Allow learners to practice NetSuite workflows without affecting real NetSuite data.
4. Explore guided simulations.
5. Explore interactive NetSuite exercises.

6. Prioritize features that demonstrate actual NetSuite learning rather than generic LMS functionality.
-

Hackathon Demo

1. Design the complete hackathon demo flow.
2. Create realistic demo data.
3. Seed demo users.
4. Seed courses.
5. Seed modules.
6. Seed quizzes.
7. Seed quiz attempts.
8. Seed learner progress.
9. Seed completed courses.
10. Seed certificates.
11. Make the dashboard look populated immediately.
12. Prepare an admin demonstration.
13. Prepare a learner demonstration.
14. Demonstrate course creation.
15. Demonstrate course learning.
16. Demonstrate video learning.
17. Demonstrate quiz failure.
18. Demonstrate quiz retry.
19. Demonstrate quiz success.
20. Demonstrate progress updates.
21. Demonstrate certificate generation.

22. Demonstrate certificate download.
 23. Demonstrate the admin dashboard.
 24. Make the entire demo possible without unnecessary manual setup.
 25. Prepare a backup recorded demo.
-

Testing

1. Start the backend and verify that Uvicorn runs.
2. Start the frontend and verify it loads.
3. Verify database connectivity.
4. Test registration.
5. Test learner login.
6. Test admin login.
7. Test JWT authentication.
8. Test admin authorization.
9. Test course creation.
10. Test course retrieval.
11. Test module creation.
12. Test module retrieval.
13. Test video display.
14. Test quiz submission.
15. Test quiz scoring.
16. Test quiz failure.
17. Test quiz retry.
18. Test quiz pass.
19. Test progress tracking.

20. Test course completion.
 21. Test certificate generation.
 22. Test certificate download.
 23. Test admin dashboard statistics.
 24. Test user management.
 25. Test the entire application end-to-end.
-

Authentication Debugging

1. Investigate the `500 Internal Server Error` occurring on:

```
POST /auth/login
```

1. Trace the login error into password verification.
2. Investigate the Passlib/bcrypt error.
3. Resolve bcrypt version compatibility.
4. Uninstall the incompatible bcrypt package.
5. Install:

```
bcrypt==4.0.1
```

1. Restart the backend.
 2. Test the admin credentials again.
 3. Verify that successful login returns a JWT.
 4. Use the JWT to access admin APIs.
-

Git/GitHub

1. Connect the project to the GitHub repository.
2. Check the configured Git remote.

3. Fetch from the origin.
 4. Resolve remote/local history conflicts.
 5. Handle the GitHub repository containing existing commits.
 6. Push the local project successfully.
 7. Verify the repository contains the current project.
 8. Keep the repository clean.
 9. Commit the final implementation.
 10. Push the final version.
-

README

1. Create a professional README for NetSuite Academy.
2. Explain what NetSuite Academy is.
3. Explain the problem being solved.
4. Explain the solution.
5. Explain the key features.
6. Explain the technology stack.
7. Explain the project architecture.
8. Explain frontend setup.
9. Explain backend setup.
10. Explain database setup.
11. Explain environment variables.
12. Explain how to run the project.
13. Explain authentication.
14. Explain learner functionality.
15. Explain admin functionality.

16. Explain quizzes.
 17. Explain progress tracking.
 18. Explain certificates.
 19. Explain AI functionality if implemented.
 20. Add demo credentials where appropriate.
 21. Add the GitHub repository information.
 22. Make the README suitable for hackathon judges.
-

AI / Prompt Documentation

1. Create documentation showing how AI was used during development.
 2. Create the prompt history for the NetSuite Academy project.
 3. Organize the prompts by development stage.
 4. Remove unnecessary conversational/debugging prompts from the condensed history.
 5. Preserve the meaningful prompts used for implementation.
 6. Create a clean prompt history that can be included in the repository.
 7. Create a README section describing AI-assisted development.
-

Final Presentation Preparation

1. Review the entire NetSuite Academy implementation.
2. Identify what is already completed.
3. Identify what is still incomplete.
4. Identify current blockers.
5. Prioritize remaining implementation work.
6. Fix critical blockers before adding optional features.
7. Verify admin login.

8. Verify learner login.
9. Verify the complete learner journey.
10. Verify certificate generation.
11. Verify certificate download.
12. Verify admin dashboard.
13. Populate real course content.
14. Polish the UI.
15. Add only the highest-value differentiating feature if time permits.
16. Run the complete end-to-end test.
17. Update README.
18. Update prompt history.
19. Commit final changes.
20. Push final changes to GitHub.
21. Prepare the final hackathon demonstration.